UN MANUALE TECNICO DI BM

Pi Zero W e bm3d

Raspberry Pi Zero W 1.1 con bm: prestazioni di ARM11 e GPU, e i driver bm3d 2D e 3D.

Indice

La scheda 4

Il processore 4

La memoria 4

I clock 5

La GPU VideoCore IV 5

Prestazioni dell'ARM11, misurate 6

Costi di base 6

Quanti oggetti stanno in un frame 6

Costo per pixel del renderer ARM 7

QEMU non è il Pi 7

Lacune nei dati dell'ARM 7

La GPU e il 2D 8

Come disegna bm in 2D 8

Risoluzioni e scaler 8

Che cosa offre gfx16 8

DMA 9

2D disegnato dalla GPU 9

La GPU e il 3D 10

Il renderer ARM (r3d) 10

Il percorso V3D (gpu3d) 10

Quando si usa l'uno o l'altro 10

Opzioni 11

Che cosa dà la GPU, misurato 12

I driver bm3d 14

Un nome, due driver 14

Le versioni che contano 14

Il lato 2D 15

Il lato 3D 15

Come sceglie un gioco 15

Misurare sulla tua scheda **17**
Il 3D Bench 17
Una parte del bench, dal monitor 17
Il costo di un pixel 18
Il test GPU e lo stress test 18
Confronta i driver sulla stessa scheda 18
Nessun report per un test fermato 18
Se mancano i numeri attuali 18

Limiti e consigli **20**
Limiti 20
Consigli 20

La RGB30, dove il repo confronta **21**

Punti aperti **22**

LEGGIMI

Che cos'è. Che cosa sa fare il Raspberry Pi Zero W 1.1 con bm, e che cosa offrono i driver di bm al tuo gioco: il processore ARM1176, la GPU VideoCore IV, il disegno 2D e i driver 3D chiamati bm3d.
Come leggerlo. Ogni numero viene da un file del repository, nominato accanto in uno span `code` come `docs/PERFORMANCE.md`. Il capitolo 6 dà i passi per misurare le stesse cose sulla tua scheda.
Che cosa manca. Ogni lacuna è segnata con un'etichetta DA COMPLETARE evidenziata ed elencata nell'ultimo capitolo. Nulla è inventato.

La scheda

Il Pi Zero W 1.1 è il primo bersaglio di bm. bm gira senza sistema operativo: il gioco, la grafica, l'audio e l'input condividono un solo core.

Il processore

Voce	Valore	Fonte
SoC	BCM2835	`docs/HARDWARE.md` (sezione 7: stesso SoC del Pi 1)
CPU	ARM1176JZF-S (ARMv6), 1 core	`docs/HARDWARE.md`
Clock	700 MHz all'avvio, 1000 MHz chiesti al firmware e misurati	`docs/HARDWARE.md`
FPU	VFPv2, double in hardware; bm attiva RunFast	`docs/HARDWARE.md`
Cache	L1 16 KiB istruzioni + 16 KiB dati; L2 128 KiB nel VideoCore	`docs/HARDWARE.md`
Divisione intera	non in hardware (una routine di libgcc): evitala nei cicli caldi	`docs/HARDWARE.md`
MMU, cache, predizione dei salti	attive, con il clock a 1000 MHz	`docs/HARDWARE.md`

La memoria

Voce	Valore	Fonte
RAM totale	512 MiB LPDDR2, condivisi da CPU e GPU	`docs/HARDWARE.md`
RAM per l'ARM	448 MiB (con `gpu_mem=64`)	`docs/HARDWARE.md`
RAM per la GPU	64 MiB; il framebuffer 640x360x4x2 ne occupa 1,8 MiB	`docs/HARDWARE.md`
Memoria per Lua	limite 64 MiB	`docs/HARDWARE.md`
Banda di copia	memcpy circa 100 MB/s, riempimento circa 430 MB/s	`docs/HARDWARE.md`, `docs/PERFORMANCE.md`
Letture dalla SDRAM (ARM)	circa 4 volte più lente delle scritture	`docs/PERFORMANCE.md`

La memoria non è il limite. Lo sono il tempo per frame e la banda verso la RAM (fonte: `docs/HARDWARE.md`).

I clock

Il repo registra questi clock da un Pi Zero W reale (fonte: `docs/bench/20261005-082943_bench3d_pi-zero-w_v0.2.0.txt`, riga `machine`):

Clock	MHz
ARM	1000
core (cache L2 e bus della memoria)	250
V3D (l'unità 3D)	250
SDRAM	400

- Il clock del core può arrivare al massimo a 400 MHz (fonte: `docs/STRESS.md`).
- `enable_uart=1` in `boot/config.txt` tiene il core a 250 MHz; anche il V3D è stato visto a 250 MHz (fonte: `docs/BENCH3D.md`).
- `v3d_clock=max` in `bm/config.txt` chiede al firmware il massimo all'avvio. È una prova, spenta di default (fonte: `docs/BENCH3D.md`).
- [DA COMPLETARE: l'effetto di `force_turbo=1` o del core a 400 MHz sul 3D Bench; `docs/STRESS.md` dice "da provare"]

La GPU VideoCore IV

Voce	Valore	Fonte
Clock del core	250 MHz	`docs/HARDWARE.md`
Unità 3D (V3D)	12 QPU in 3 slice, texture filtrate, tile 64x64 con z a 24 bit sul chip	`docs/HARDWARE.md`
Picco secondo Raspberry Pi	1 Gpixel/s e 1,5 Gtexel/s	`docs/BENCH3D.md`
Scaler (HVS)	ingrandisce il framebuffer al modo della TV senza costo per la CPU	`docs/progress/graphics.md`
Uscita	mini-HDMI fino a 1920x1080 a 60 Hz	`docs/HARDWARE.md`
Vsync del firmware	non disponibile sul Pi Zero; bm scandisce i 60 fps con il suo timer	`docs/HARDWARE.md`

bm non usa OpenGL ES. Scrive da sé i registri del V3D, con un disegno a funzioni fisse e i suoi shader per le QPU (fonti: `docs/PERFORMANCE.md`, sezione 7, per il driver a funzioni fisse con registri diretti; `docs/DRIVERS.md` per gli shader).

[DA COMPLETARE: shots/pi-zero-w-bench-score.png]

Prestazioni dell'ARM11, misurate

Tutti i numeri qui sotto sono stati misurati su un Pi Zero W reale, salvo dove il testo dice altro. Il budget per frame a 60 fps è 16,7 ms e a 30 fps è 33,3 ms (fonte: `docs/PERFORMANCE.md`).

Costi di base

Fonte: `docs/PERFORMANCE.md`, sezione 2, e `docs/HARDWARE.md`.

Operazione	Costo
Riempimento 640x360, 32 bit	2,2 ms
Copia di 450 KiB da RAM a RAM, CPU / DMA	3,70 / 2,99 ms
Riempimento di 450 KiB in RAM, CPU / DMA	1,02 / 0,85 ms
Striscia di 48 righe da RAM allo schermo, CPU / DMA	0,87 / 0,26 ms
Mappa + 256 sprite, diretto / via un buffer in RAM (benchmark `p`)	8,58 / 11,85 ms
Copia dell'intero frame via RAM, con il DMA	14,06 ms (più lenta della CPU)
256 sprite 16x16	circa 0,9 ms
Una semplice operazione Lua	circa 100 ns (circa 140.000 per frame)
Lua `fib(25)` / 1 milione di somme	83 ms / 104 ms

Quanti oggetti stanno in un frame

Lo stress test conta gli oggetti disegnati per frame a 640x360, RGB565. Fonte: `docs/STRESS.md`, misura del 2026-10-05, righe ARM (il rasterizzatore ARM, bm3d 0.2).

Test	60 fps	30 fps	Costo per oggetto
sprite 16x16 (C)	4454	9200	3,51 us
sprite 32x32 (C)	1501	3108	10,37 us
triangoli 2D (circa 170 px)	4726	9777	3,29 us
sfere 3D di 96 triangoli (C)	63 (2451 triangoli)	201 (7910)	128,01 us
sfere 3D, Gouraud	25 (990 triangoli)	99 (3846)	212,54 us
sfere 3D, con texture	meno di 1	50 (1910)	259,37 us
quad 320x180, piatto	11	25	22 ns per pixel
quad 320x180, piatto senza z	52	113	5 ns per pixel
quad 320x180, Gouraud	6	13	41 ns per pixel
quad 320x180, con texture	3	6	81 ns per pixel
sprite 16x16 (Lua)	1990	4110	7,85 us

Che cosa dicono i numeri (fonte: `docs/STRESS.md`):

- Uno sprite costa quasi solo i suoi pixel: il 16x16 costa circa 14 ns per pixel, e il 32x32 ha 4 volte i pixel ma costa 3 volte tanto. Il limite è la banda verso la memoria video.
- Lua costa circa 5 us in più per sprite rispetto al C (8,5 contro 3,5 nella misura di settembre): il calcolo della posizione e la chiamata `spr()`.
- Un triangolo 2D pieno di circa 170 pixel costa 6,0 us nella misura di settembre (`docs/PERFORMANCE.md`).

Costo per pixel del renderer ARM

Fonte: `docs/PERFORMANCE.md`, sezione 5. Una sfera a schermo intero è circa 180.000 pixel.

Modo	Risultato sul Pi
Piatto	sta in 16,7 ms: 60 fps con 15 sfere
Gouraud	oltre 16,7 ms: più di circa 90 ns per pixel
Con texture	oltre 33 ms: più di circa 180 ns per pixel

Regola pratica dello stesso file: i triangoli Gouraud e con texture vanno bene per geometria piccola e media, non per riempire lo schermo. Disegna pavimenti e cieli grandi piatti e senza z-buffer.

QEMU non è il Pi

QEMU va circa 2 volte più piano del Pi negli sprite (2133 contro 4482 a 60 fps) e nel 3D piatto (602 contro 1195 triangoli). Non modella la cache né il bus della memoria, quindi Gouraud e texture lì sembrano molto meglio che sulla scheda vera (fonte: `docs/PERFORMANCE.md`, sezione 6). Conferma sempre una velocità sulla scheda.

Lacune nei dati dell'ARM

- **[DA COMPLETARE: il costo in cicli della divisione intera (libgcc) sull'ARM1176; il repo dice solo che non è in hardware]**

La GPU e il 2D

Come disegna bm in 2D

- Tutto il 2D è codice C sull'ARM; Lua dà solo la logica (fonte: `docs/progress/graphics.md`).
- Le cartucce disegnano in RGB565 (16 bit). La console e il menu usano 32 bit (fonte: `docs/HARDWARE.md`). L'RGB565 dimezza i byte per pixel; a 32 bit il tempo di copia di un frame raddoppia, da circa 6,5 ms a circa 13 ms con la CPU (fonte: `docs/PERFORMANCE.md`).
- Il framebuffer viene dalla mailbox del firmware. La sua altezza è l'altezza dello schermo per il numero di pagine (da 1 a 3). Il cambio di pagina modifica l'offset virtuale (fonte: `docs/progress/graphics.md`).
- Di default le cartucce disegnano direttamente nella memoria video, perché copiare dalla RAM rilegge la lenta SDRAM: 8,58 ms diretto contro 11,85 ms via RAM (fonte: `docs/PERFORMANCE.md`). *Settings* > *Screen and sound* > *Game drawing (.bm)* passa da `Direct` a `Via RAM` (fonte: `src/kernel/settings.c`).
- bm usa il triplo buffer, che ha eliminato lo strappo nella metà bassa del menu (fonte: `docs/PERFORMANCE.md`).
- La griglia di luce 4x4, bilineare e con dither, è usata dai giochi con luci dinamiche, a 320x180 (fonte: `docs/PERFORMANCE.md`).

Risoluzioni e scaler

Le risoluzioni delle cartucce sono 640x360 (predefinita), 480x270 e 320x180, più i quadrati 256x256 e 360x360 dell'edizione `.b16`. `screen(w, h)` cambia a run time fra 320x180, 384x216, 480x270, 640x360, 960x540, 1280x720 e 1920x1080 (fonti: `docs/progress/graphics.md`, `docs/api/draw.md`).

Lo scaler HVS ingrandisce l'immagine fino alla TV. `scaling_kernel` in `config.txt` (nella radice della SD) sceglie il suo filtro (fonte: `docs/progress/v3d.md`):

Valore	Filtro	Stato nel repo
8	nearest neighbour	verificato sul Pi, quello che bm usa oggi
6	`SINC_HAMMING_3PI`, il predefinito del firmware	verificato sul Pi: bello, morbido, da 640x360 a 1080p
da 1 a 5, 7	altre varianti sinc	elencate dal firmware, non provate sul Pi

- [DA COMPLETARE: come appaiono sul Pi i valori 3, 5 e 7 di `scaling_kernel`; `docs/progress/v3d.md` li elenca come non provati]

Che cosa offre gfx16

La libreria 2D è `src/bm/gfx16.h`. Ogni chiamata è ritagliata nel rettangolo di clip e spostata dalla camera (fonte: `src/bm/gfx16.h`):

Gruppo	Chiamate (nome C; l'API Lua è in `docs/api/draw.md`)
Forme	`g16_cls`, `g16_pset`, `g16_pget`, `g16_line`, `g16_rect`, `g16_rectfill`, `g16_circ`, `g16_circfill`
Sprite	`g16_spr`, `g16_sspr`, `g16_sspr_zoom` (pixel più vicino, più grande o più piccolo)
Mappa	`g16_map`, `g16_map_mask` (solo le celle la cui tile ha un flag)
Testo	`g16_text`, `g16_text_scaled`
Luce	`g16_light_*` (griglia ogni 4 pixel), `g16_fade_*` (luce a livelli)
Sheet	`g16_sheet_alloc`, `g16_sheet_set`, `g16_sheet_update_cell`

Gli sprite sheet arrivano a 4096x4096 pixel. Uno sheet con al massimo 256 colori può essere salvato come SHEET8 (palette + sequenze), molto più piccolo nella cartuccia (fonti: `docs/api/draw.md`, `docs/PERFORMANCE.md`).

Le primitive a 32 bit della console sono in `src/gfx/draw.h`: `gfx_clear`, `gfx_rect`, `gfx_sprite16` (16x16, un bit) e `gfx_text`. Disegnano sul framebuffer, e i giochi non le usano.

DMA

- Il DMA serve per l'audio HDMI e per le copie di frame quando sono attive (fonte: `docs/HARDWARE.md`).
- Per il 2D il DMA vince sulle strisce e sui riempimenti (striscia di 48 righe: 0,26 ms contro 0,87 ms con la CPU) e perde su un frame intero (14,06 ms contro 11,85 ms). bm copia i frame interi con la CPU (fonte: `docs/PERFORMANCE.md`).
- Le impostazioni del DMA sono prudenti di proposito: burst 8, priorità alta, `WAIT_RESP`. Quelle aggressive bloccavano il Pi (fonte: `docs/PERFORMANCE.md`).
- A fine frame lo z-buffer del 3D viene azzerato dal DMA mentre gira il prossimo `_update`, circa 1 ms risparmiato a 640x360. `dma_zclear=0` lo spegne (fonte: `docs/PERFORMANCE.md`).

2D disegnato dalla GPU

Con `gpu3d_2d=1` (spento se la chiave manca) la GPU disegna il 2D che viene dopo il 3D nello stesso job: `rect`, `pset`, linee orizzontali e verticali, `spr`, `sspr` (anche specchiati e con zoom intero), le celle di `map()` e il testo di `print`. I pixel sono gli stessi dell'ARM. Cerchi, linee oblique, triangoli, `prompt` e uno zoom non intero chiudono il job, e l'ARM li disegna come prima (fonti: `docs/DRIVERS.md`, `src/kernel/settings.c`).

Misurato sul Pi: il 2D sulla GPU costa 9,5 ns per pixel, 19,6 ms a 1080p (fonte: `docs/progress/v3d.md`).

La GPU e il 3D

bm ha due modi di disegnare il 3D, dietro le stesse funzioni. La tua cartuccia non cambia.

Il renderer ARM (r3d)

`src/bm/r3d.c` gira sull'ARM. Trasforma, illumina, scarta e taglia con float VFP e rasterizza in virgola fissa in uno z-buffer a 16 bit (1/z). Offre (fonte: `docs/progress/graphics.md`):

- ombreggiatura piatta e Gouraud (con dither 4x4), texture con prospettiva corretta dallo sheet, taglio sul piano vicino;
- luce del sole e del cielo/terreno, speculare, rim, fino a 4 lampade puntiformi, nebbia;
- bit di materiale per faccia: emissivo, lucido, retino, livello di dettaglio;
- ombre planari, un livello in prima persona, punti, linee e billboard;
- skinning rigido (un osso per vertice).

È il driver bm3d 0.2. Disegna tutto quando `gpu3d=0`, in QEMU, quando la GPU non risponde o non supera la prova all'avvio, e per quello che la GPU non sa fare (fonte: `docs/progress/graphics.md`).

Il percorso V3D (gpu3d)

`src/gpu/gpu3d.c` è un backend di r3d. L'ARM trasforma, illumina e taglia ancora; il V3D riempie i pixel (fonte: `src/gpu/gpu3d.h`, `docs/GPU-BENCHMARK-REPORT.md`).

- Il V3D disegna a tile di 64x64 pixel: una lista di binning smista i triangoli per tile, poi ogni tile viene disegnata sul chip (colore, profondità a 24 bit) e salvata (fonte: `src/gpu/v3d.h`).
- r3d raccoglie i triangoli in un solo job GPU per frame, dove possibile. Le tile sono caricate dalla pagina, quindi il 2D disegnato prima del 3D resta sotto.
- Gli shader sono piccoli programmi per le QPU scritti in `tools/qpuasm.py` e generati in `src/gpu/shaders.h`.
- Differenze dal renderer ARM: Gouraud liscio senza rumore di dither, profondità a 24 bit invece di 1/z a 16 bit, nessun bisogno di `zclear()` per il buffer di profondità (parte da zero a ogni frame) (fonti: `docs/GPU-BENCHMARK-REPORT.md`, `docs/api/light3d.md`).
- Un vertex shader (attivo di default) può piazzare i vertici di intere mesh sulla GPU, così l'ARM manda solo matrici e luci.
- I pixel coincidono con quelli del renderer ARM entro lo 0,1% nei test sull'emulatore (fonte: `docs/GPU-BENCHMARK-REPORT.md`).

Quando si usa l'uno o l'altro

Situazione	Chi disegna
Un V3D risponde e `gpu3d` non è 0	la GPU (predefinito)
`gpu3d=0` in `bm/config.txt`, o *Settings* > *Screen and sound* > *3D of the games: ARM*	l'ARM
QEMU	l'ARM
La GPU va in timeout o una prova all'avvio fallisce	l'ARM subentra da solo, con una riga nel log
2D che la GPU non sa riprodurre esattamente (elenco sopra)	l'ARM, dopo il job

Situazione	Chi disegna
Un triangolo che attraversa la guard band dei vertici	tagliato dall'ARM

Fonti: `docs/progress/graphics.md` , `docs/GPU-BENCHMARK-REPORT.md` , `docs/DRIVERS.md` .

Le funzioni opzionali sono provate all'avvio. La prova controlla che la GPU disegni quello che disegnerebbe l'ARM, e spegne da sola la funzione se non è così. La riga di stato (*Dev* > *GPU test*, o `g` nel monitor) dice che cosa è stato tenuto, per esempio `vertex shader yes` o `ETC1 textures yes (utiles 2x4, bytes BE)` (fonte: `docs/DRIVERS.md`).

Opzioni

Ogni opzione è una chiave in `bm/config.txt` e una riga in *Settings* > *Screen and sound*. Il predefinito è quello che vale quando la chiave manca (fonte: `src/kernel/settings.c` , `docs/DRIVERS.md`).

Chiave	Che cosa fa	Predefinito
`gpu3d`	la GPU disegna il 3D (0 = ARM)	attivo dove risponde un V3D
`gpu3d_aa`	MSAA 4x	spento
`gpu3d_vs`	vertex shader: 0 ARM, 1 scenario, 2 tutti i modelli	2
`gpu3d_queue`	1: il job parte e il prossimo `_update` gira intanto; 2: due job in volo	spento
`gpu3d_later`	ogni frame mostrato un frame dopo (richiede `gpu3d_queue=2`)	spento
`gpu3d_2d`	2D sopra il 3D dentro il job GPU	spento
`gpu3d_filter`	texture bilineari (l'aspetto cambia)	spento
`gpu3d_wc`	memoria dei job non in cache	spento
`gpu3d_tex16`	texture a 16 bit (RGB565; sheet con ritagli come RGBA5551)	spento
`gpu3d_etc1`	sheet opachi come ETC1 (imposta anche `gpu3d_tex16`)	spento
`gpu3d_fs2`	pixel shader con texture a due thread	attivo (`0` lo spegne)
`gpu3d_sort`	mesh dalla più vicina, triangoli nell'ordine della cache dei vertici	spento
`gpu3d_attr`	attributi dei vertici compatti	spento
`r3d_fast`	renderer ARM: una matrice e la luce nello spazio del modello (i pixel non sono più identici bit a bit)	`r3d_fast=1` per attivarlo
`r3d_impostor`	modelli lontani come un solo quad (da 2 a 64 pixel di uno schermo di 1080 righe)	spento

Nelle Settings le righe si chiamano *3D of the games*, *3D anti-aliasing*, *3D vertices*, *3D frame queue*, *3D frame shown*, *2D over the 3D*, *3D textures*, *3D job memory*, *3D pixel shaders*, *3D draw order*, *3D corners*, *3D impostors* e *3D on the ARM* (fonte: `src/kernel/settings.c`).

Limiti che incontri:

- L'MSAA funziona solo nei job senza uno z-buffer conservato. Una cartuccia che disegna 3D, poi 2D, poi di nuovo 3D nello stesso frame resta senza anti-aliasing (fonte: `docs/api/light3d.md`).
- Le texture con i lati multipli di 32 vanno alla GPU in T-format, più veloce da leggere (fonte: `docs/api/light3d.md`).
- L'ETC1 richiede sheet con i lati multipli di 64x128 (o 128x64) texel (fonte: `docs/DRIVERS.md`).
- Sul Pi, gli sheet a 16 bit e ETC1 hanno dato la stessa velocità dei 32 bit nel 3D Bench, entro l'1-6%, perché gli sheet piccoli stanno nella cache delle texture (fonte: `docs/progress/v3d.md`).
- Disegna prima tutto il 3D e poi l'HUD. Il 3D disegnato dopo il 2D nello stesso frame costringe la GPU a conservare lo z-buffer, circa 1 MB scritto e riletto a ogni frame (fonte: `docs/api/light3d.md`).

Che cosa dà la GPU, misurato

Fonte: `docs/GPU-BENCHMARK-REPORT.md`, sezione 4, con ARM 1000 MHz, core 250, V3D 250, SDRAM 400.

Misura	Risultato
Triangoli piccoli	3,0 milioni al secondo (20.000 preparati in 7,7 ms)
Fill rate	811 Mpixel/s (il riempimento software è circa 45 Mpixel/s)

Stress test, il massimo di oggetti per frame a 640x360:

Scena	ARM, 60 fps	GPU, 60 fps	GPU, 30 fps
Sfere piatte	69 (2679 triangoli)	182 (7142)	389 (15346)
Sfere Gouraud	18 (703)	170 (6673)	364 (14360)
Sfere con texture	meno di 1	156 (6094)	333 (13135)
Quad piatti a schermo intero	11 (22 ns/px)	200 (1 ns/px)	430
Quad Gouraud a schermo intero	5 (49 ns/px)	199 (1 ns/px)	427

Programmi veri, stesso file:

Programma	ARM	GPU
Texture Room HD 640x360	25,3 ms (8 casse)	5,8 ms (32 casse, 573 triangoli)
Chaos Kitchen	14,1 ms	6,8 ms (758 triangoli)

Calano anche le istruzioni ARM per frame, in una simulazione host QEMU (non misurate sul Pi): Texture Room, 8 casse, 320x180, 446 triangoli, passa da 3,86 milioni a 0,62 milioni (fonte: `docs/GPU-BENCHMARK-REPORT.md`, sezione "Host Simulation Metrics (qemu-arm)").

Con la GPU le sfere sono limitate dall'ARM (circa 2 us per triangolo fra trasformazioni e vertici), mentre i quad sono limitati dal V3D (fonte: `docs/STRESS.md`). La misura del 2026-10-05 (bm3d 4.2, stesso file) ha dato 207 sfere sulla GPU a 60 fps (8131 triangoli), 159 Gouraud, 165 con texture, e un quad con texture 256x256 a 3 ns per pixel (91 quad), con l'MSAA 4x che costa quasi niente (204 sfere).

Un gioco vero: Overbit

Overbit è il titolo 3D più esigente di bm. Fonte: `docs/GPU-BENCHMARK-REPORT.md`, sezione 6 (stime per uno scontro con 10 bot sul Pi Zero W):

Preset	ARM	GPU
LOW	36,0 fps	49,6 fps
MEDIUM	33,7 fps	45,0 fps
HIGH	29,8 fps	35,5 fps
ULTRA	25,1 fps	33,8 fps
EXTREME	24,7 fps	31,0 fps

La GPU riduce le istruzioni ARM per frame dal 23% al 32% in questi preset. A 1080p una partita giocata è limitata dal riempimento: la GPU impiega 20,4 ms per 6105 triangoli e 10,1 ms a 720p. La leva sono meno pixel, non meno triangoli (fonte: `docs/progress/v3d.md`).

- [DA COMPLETARE: Overbit giocato sul Pi con `gpu3d_attr=1`, `gpu3d_tex16=1` e `gpu3d_etc1=1` contro tutto spento; `docs/progress/v3d.md` lo elenca come ancora da misurare]

I driver bm3d

Un nome, due driver

Il 3D passa per **r3d** (scena, trasformazioni, luce, il rasterizzatore ARM) e **gpu3d** (il backend V3D con i suoi shader). Condividono una sola versione, **bm3d X.Y**. L'attuale è bm3d 7.3 (fonte: `src/gpu/version3d.h`). La vedi in *Settings* > *System* > *3D driver*, nel test GPU, nel log quando parte un gioco, nel 3D Bench e come quarto valore di `gpu3d()` (fonte: `docs/DRIVERS.md`).

Le versioni che contano

Fonte: `docs/DRIVERS.md`.

Versione	Che cosa ha portato
0.2	solo ARM: bordi in virgola fissa, cicli di texture specializzati, z azzerato dal DMA (69 sfere a 60 fps)
1.0	la GPU disegna per la prima volta i triangoli (shader NV)
2.0	pagine pulite, texture in formato T e MSAA (182 sfere a 60 fps)
2.1	Overbit sulla GPU: facce a retino, luce RGB e nebbia precalcolate, ombre, effetti 3D
da 3.0 a 3.4	il vertex shader piazza lo scenario, poi i modelli illuminati, le ossa e le skin
4.0, 4.1	il frame in coda: il job parte e il prossimo `_update` gira mentre la GPU disegna
4.2	schermi fino a 1920x1080; `cls()` fatto dal job GPU
5.0	mesh fino a 65535 vertici e 65535 facce, indicizzate sulla GPU
5.1	due job in volo
5.2, 7.2	texture a 16 bit (sheet opachi, poi quelli con ritagli)
5.3, 6.4	shader con texture a due thread, attivi di default dalla 6.4
5.4, 5.6	ordine di disegno, dal più vicino e secondo la cache dei vertici
5.5	`visible3d()` e `pvs3d()`
6.6	il vertex shader attivo di default per tutti i modelli
6.9	ogni frame mostrato un frame dopo (opzione)
7.0	facce con texture disegnate a colore piatto (flag 128 di `draw3d`, `textures3d(false)`)
7.1	attributi compatti
7.3	sheet opachi come ETC1

`docs/DRIVERS.md` spiega anche come le impostazioni riproducono le versioni precedenti sul codice di oggi, così puoi confrontarle sullo stesso Pi: `gpu3d=0` è la 0.2, `gpu3d_vs=0` è la 2.1, `gpu3d_vs=1` è la 3.0, `gpu3d_vs=2` è la 3.4, e così via. Le versioni 0.1 e 1.0 non girano più; il repo conserva i numeri misurati con esse.

Il lato 2D

Per un gioco il driver 2D è `gfx16`, con le chiamate di `docs/api/draw.md`: `cls`, `screen`, `pset`, `pget`, `line`, `rect`, `rectfill`, `circ`, `circfill`, `tri`, `print`, `font`, `prompt`, `camera`, `clip`, `spr`, `sspr`, `zspr`, `map`. `cls()` con il 3D sulla GPU è gratis: il job GPU azzera la pagina a quel colore. Se il 2D viene prima del 3D, l'ARM la riempie come sempre (fonte: `docs/api/draw.md`).

Il lato 3D

Le funzioni sono descritte in `docs/api/light3d.md`. Quelle che contano per le prestazioni:

Funzione	Uso
`mesh(v, f, [uv])`, `model(name)`	mesh, fino a 65535 vertici e 65535 facce
`draw3d(m, x, y, z, [rx, ry, rz, scale, flags])`	flag: 1 senza z-buffer (pavimenti, cieli: più veloce), 2 senza luce, 4 liscio, 8 ombra, 64 davanti, 128 texture piatte; si sommano
`camera3d`, `light3d`, `sky3d`, `fog3d`, `lamp3d`	camera e luce
`zclear()`	azzera lo z-buffer
`visible3d(x, y, z, r)`, `pvs3d(t)`	salta quello che non si vede prima di disegnarlo
`textures3d(false)`	ogni `draw3d` disegna le facce con texture a colore piatto
`stat(9)`	1 quando il 3D lo fa la GPU
`stat(16)`	il tempo del job GPU dell'ultimo frame in ms; letto con la coda attiva include l'attesa, quindi fidati solo senza la coda

Come sceglie un gioco

Senza fare nulla, il gioco riceve le impostazioni della console. Un gioco può chiedere le sue con `gpu3d([on, aa, vs, queue, impostor, later])`. Restituisce `on, aa, vs, version, queue, impostor, later`, e cambia la scelta solo per questa cartuccia (fonte: `docs/api/light3d.md`):

- `on` resta `false` se non c'è GPU (QEMU, `gpu3d=0`) e nei giochi 256x256 (la GPU scrive pagine intere);
- `aa` resta spento se l'MSAA non si può usare, `vs` resta spento se la prova all'avvio non ha visto funzionare il vertex shader;
- `queue true` chiede il livello della console, `false` nessuno; `later` funziona solo con `queue 2` e mai in una partita giocata online;

- `version` è la versione di bm3d che questa scelta riproduce (`"0.2"` per l'ARM, `"2.1"` , `"3.0"` , `"3.4"` , `"4.1"` , `"6.9"`).

DA COPIARE

```
-- chiedi la GPU con il vertex shader per ogni modello
gpu3d(true, false, 2)
-- impostor solo per i modelli lontani, il resto resta com'è
gpu3d(nil, nil, nil, nil, 16)
-- leggi che cosa è in vigore
local on, aa, vs, version = gpu3d()
```

Misurare sulla tua scheda

Fallo su un Pi Zero W. Puoi eseguire tutto dal menu e leggere i report sulla SD; non serve il cavo seriale. Per mettere un kernel nuovo sulla SD copi lì `dist/kernel.img`, con `./easy_install.sh` (fonte: `docs/BENCH3D.md`, `docs/STRESS.md`).

Il 3D Bench

È l'unico benchmark del 3D di bm. Ogni test fa crescere il carico finché un frame supera i 40 ms. Esegue ogni profilo di driver (ARM, GPU, GPU+AA, GPU+VS, GPU+VS+FS2 e altri) e interpola i carichi a 60 fps e a 30 fps. Fonte: `docs/BENCH3D.md`.

Provalo, passo per passo

1. Metti la SD nel Pi Zero W e accendilo. Aspetta il menu di bm.
2. Apri la scheda Dev e scegli **3D Bench**. Puoi anche premere `j` nel monitor seriale.
3. Lascialo in pace. Il report del Pi del 2026-10-05 finisce con `total 79 s`. Ogni passo misura 6 frame e tiene quello di mezzo, quindi un singolo frame lento non sposta il risultato.
4. Usa sinistra e destra per cambiare pagina. La prima pagina è il punteggio. Premi B (o Esc) per uscire.
5. Estrai la SD e apri `bm/bench/`: la prova ha salvato `3D0001.TXT`, `3D0002.TXT`, e così via.
6. La prova successiva legge l'ultimo report e si confronta con quello. Per conservare uno storico, copia i file in `docs/bench/` del repository.

PROVA

Esegui il 3D Bench due volte. Alla seconda prova le pagine mostrano tacche bianche ai valori della prima, e una colonna "x last".

[DA COMPLETARE: shots/pi-zero-w-bench-test.png]

Che cosa leggi (fonte: `docs/BENCH3D.md`):

- Il **punteggio**: 1000 è bm3d 2.1 sul Pi Zero W, 2000 è un driver due volte più veloce in ogni test.
- I **triangoli per frame a 60 fps** della scena `mix`, con il driver dei giochi.
- Per ogni test e profilo: il carico a 60 fps, i ms (medio e peggiore), i triangoli, i ms della GPU, e le istruzioni ARM e i miss della cache dati per frame (dai contatori dell'ARM1176, solo sul Pi vero).
- La riga `machine` con i clock, la temperatura e i flag di throttling.

Una parte del bench, dal monitor

La riga del monitor esegue solo alcuni test e profili (fonte: `docs/BENCH3D.md`).

DA COPIARE

```
:gpu; b3d tests=match,quad_tex profiles=GPU,FS2; send
```

Dal PC, con la scheda in rete:

DA COPIARE

```
./easy_install.sh line "gpu; b3d; send"
```

La console scrive quello che fa finché non dice `the line is done`.

Il costo di un pixel

Fonte: `docs/progress/v3d.md` e `docs/BENCH3D.md`. Le righe `P` del report danno i ms per riempire 640x360, 1280x720 e 1920x1080.

DA COPIARE

```
./easy_install.sh line "gpu; b3d tests=quad_flat,quad_tex,bilinear,gpu2d profiles=ARM,GPU,FS2,2D,VSF; se
```

Il test GPU e lo stress test

1. Apri la scheda Dev e scegli **GPU test** (o `g` nel monitor). Esegue i passi del driver, stampa la riga di stato e il log delle prove, e riporta la velocità in triangoli e il fill rate (fonti: `docs/DRIVERS.md`, `docs/GPU-BENCHMARK-REPORT.md`).
2. Per lo stress test di sprite e 3D, costruisci sul PC il kernel dedicato e copialo sulla SD (fonte: `docs/STRESS.md`):

DA COPIARE

```
cd ~/bm && git pull
make sdcard-stress
cp dist/kernel.img /path/to/the/SD/ && sync
```

3. Avvia il Pi. Dopo il riepilogo di sistema girano i test, circa 30-60 s in tutto, e la tabella dei risultati resta sullo schermo: basta una foto.
4. Per tornare al kernel normale, esegui `make sdcard` e copia di nuovo `dist/kernel.img`.

Se hai un cavo seriale, lo stress test parte dal monitor con il tasto `S`.

Confronta i driver sulla stessa scheda

Le impostazioni riproducono i driver precedenti (vedi il capitolo 5), quindi puoi eseguire lo stesso bench con ciascuno. Usa `set key=value` nel monitor per cambiare una chiave di `bm/config.txt` fino al prossimo riavvio e `save` per tenerla (fonte: `docs/BENCH3D.md`). Ogni report stampa le opzioni 3D in vigore su una riga `options`, per esempio `bm3d 7.3 gpu3d=on vs=2 aa=0 queue=2 wc=0 filter=0 tex16=0 fs2=1 sort=0 attr=0 etc1=0`.

ATTENZIONE

Non confrontare numeri di kernel con righe `options` diverse, o con clock diversi nella riga `machine`. Il repo ha visto `quad_flat` valere 175, 91 e 70 in prove diverse con lo stesso driver, e lo ha legato ai clock del core e del V3D (fonte: `docs/BENCH3D.md`).

Nessun report per un test fermato

Un test che fermi (Start+Select, Ctrl+Esc o PS dopo il frame sullo schermo) non lascia nessun report (fonte: `docs/progress/graphics.md`).

Se mancano i numeri attuali

- [DA COMPLETARE: un report completo del 3D Bench del Pi Zero W a bm3d 7.3; l'unico report completo in `docs/bench/` è a bm3d 4.2 (kernel v0.2.0)]

Limiti e consigli

Limiti

- Un solo core fa tutto: gioco, grafica, audio, input. Il budget per frame è 16,7 ms a 60 fps (fonte: `docs/PERFORMANCE.md`).
- La banda di memoria è il primo collo di bottiglia, non la dimensione della RAM. Ogni byte in più per pixel costa (fonte: `docs/PERFORMANCE.md`).
- Il 3D sulla GPU è limitato dall'ARM per i triangoli piccoli (circa 2 us per triangolo) e dal V3D per quelli grandi (fonte: `docs/STRESS.md`).
- Divisioni: usale al massimo per triangolo, mai per scanline o per pixel. Un float denormale manda in trap se RunFast non è attivo; bm lo attiva all'avvio (fonte: `docs/PERFORMANCE.md`).
- Il vsync del firmware non è disponibile sul Pi Zero (fonte: `docs/HARDWARE.md`).
- Una porta USB OTG con un dispositivo HID più un mouse; fino a 4 DualShock 4 via Bluetooth (fonte: `docs/HARDWARE.md`).
- Il 3D non è OpenGL: è il driver a funzioni fisse di bm. Eseguire programmi tuoi sulle QPU (particelle, effetti a schermo intero) è elencato come non iniziato (fonte: `docs/progress/v3d.md`).
- Nessun interrupt per il V3D: il driver fa polling (fonte: `docs/progress/v3d.md`).

Consigli

- Scegli prima la risoluzione. 640x360 è l'11% dei pixel del 1080p; 480x270 è una scala intera esatta di 4 volte fino al 1080p (fonti: `docs/HARDWARE.md`, `docs/GPU-BENCHMARK-REPORT.md`). Sopra 640x360 sul Pi il rasterizzatore ARM paga ogni pixel e serve la GPU (fonte: `docs/api/draw.md`).
- Disegna pavimenti e cieli con il flag 1 di `draw3d` (senza z-buffer).
- Disegna prima tutto il 3D, poi l'HUD.
- Inizia il frame con `cls()`: con la GPU la pagina non viene riletta.
- Usa `visible3d()` per saltare i personaggi nascosti.
- Usa `r3d_impostor` per le folle di modelli lontani.
- Usa sheet con i lati multipli di 32, così la GPU li legge in T-format.
- Prova sulla scheda. QEMU non modella la cache né il bus della memoria (fonte: `docs/PERFORMANCE.md`).
- Quando i numeri saltano da una prova all'altra, leggi la riga `machine`: il repo ha legato i quad lenti ai clock del core e del V3D (fonte: `docs/BENCH3D.md`).

La RGB30, dove il repo confronta

La PowKiddy RGB30 (RK3566, AArch64) esegue lo stesso 3D Bench. Il suo report nel repo è `docs/bench/20261005_bench3d_rgb30_v0.2.1.txt`, bm3d 4.2, e ha solo il profilo ARM: "no GPU driver on the RGB30 yet". Il report del Pi è `docs/bench/20261005-082943_bench3d_pi-zero-w_v0.2.0.txt`, stesso bm3d. Il profilo ARM (bm3d 0.2) carica a 60 fps:

Test	Pi Zero W, ARM	RGB30, ARM
spheres	72,3	180,1
spheres_smooth	21,9	100,3
heroes	3,9	8,6
draws	453,1	899,9
quad_flat	11,7	16,2
quad_tex	3,4	8,0

Un valore -1 significa che già il primo passo non stava in 16,7 ms: `spheres_tex` sul Pi è -1 e 45,4 sulla RGB30; `match` è -1 sul Pi e 1,3 sulla RGB30.

Da allora il repo ha avviato un driver per la GPU Mali-G52 della RGB30. Accende la GPU, esegue job e azzera una superficie; un triangolo è scritto e non ancora provato sul dispositivo. Il 3D della RGB30 è ancora `r3d` sull'ARM (fonte: `docs/progress/graphics-mali.md`). Quindi sulla RGB30 non ci sono numeri GPU da confrontare con il V3D del Pi.

Punti aperti

- **La scheda**: l'effetto di `force_turbo=1` o del core a 400 MHz sul 3D Bench; `docs/STRESS.md` dice "da provare"
- **La scheda**: immagine da fare: shots/pi-zero-w-bench-score.png
- **Prestazioni dell'ARM11, misurate**: il costo in cicli della divisione intera (libgcc) sull'ARM1176; il repo dice solo che non è in hardware
- **La GPU e il 2D**: come appaiono sul Pi i valori 3, 5 e 7 di `scaling_kernel`; `docs/progress/v3d.md` li elenca come non provati
- **La GPU e il 3D**: Overbit giocato sul Pi con `gpu3d_attr=1`, `gpu3d_tex16=1` e `gpu3d_etc1=1` contro tutto spento; `docs/progress/v3d.md` lo elenca come ancora da misurare
- **Misurare sulla tua scheda**: immagine da fare: shots/pi-zero-w-bench-test.png
- **Misurare sulla tua scheda**: un report completo del 3D Bench del Pi Zero W a bm3d 7.3; l'unico report completo in `docs/bench/` è a bm3d 4.2 (kernel v0.2.0)